Lathika N 241801135

exp 5

```
!pip install -q spacy
!python -m spacy download en_core_web_sm -q

import pandas as pd
import spacy
import os
from google.colab import files

if not os.path.exists('/content/Reviews.csv'):
    print("Please upload the Reviews.csv dataset")
    uploaded = files.upload()

    for filename in uploaded:
        if filename.endswith('.csv'):
            os.rename(filename, '/content/Reviews.csv')
            break

nlp = spacy.load("en_core_web_sm")

df = pd.read_csv('/content/Reviews.csv')

print("Dataset loaded successfully!")
print("Shape:", df.shape)
display(df.head())
```

```
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 12.8/12.8 MB 78.7 MB/s eta 0:00:00
✓ Download and installation successful
You can now load the package via spacy.load('en_core_web_sm')
⚠ Restart to reload dependencies
If you are in a Jupyter or Colab notebook, you may need to restart Python in
order to load all the package's dependencies. You can do this by selecting the
'Restart kernel' or 'Restart runtime' option.
Dataset loaded successfully!
Shape: (27783, 10)
```

	Id	ProductId	UserId	ProfileName	HelpfulnessNumerator	HelpfulnessDenominator	Score	Time	Summ
0	1	B001E4KFG0	A3SGXH7AUHU8GW	delmartian	1	1	5	1303862400	G Qu Dog F
1	2	B00813GRG4	A1D87F6ZCVE5NK	dll pa	0	0	1	1346976000	No Adverti
2	3	B000LQOCH0	ABXLMWJIXXAIN	Natalia Corres "Natalia Corres"	1	1	4	1219017600	"Deli says i
3	4	B000UA0QIQ	A395BORC6FGVXV	Karl	3	3	2	1307923200	Co Medi
4	5	B006K2ZZ7K	A1UQRSCLF8GW1T	Michael D. Bigham "M. Wassir"	0	0	5	1350777600	Great t

```
# Select the review text column
df = df[['Text']]

# Remove missing values
df = df.dropna(subset=['Text'])
```

```
# Remove duplicate reviews
df = df.drop_duplicates(subset=['Text'])

# Keep the first 10,000 reviews
df = df.head(10000).reset_index(drop=True)

# Display the cleaned dataset
print("Shape after cleaning:", df.shape)
print("Missing values:", df['Text'].isnull().sum())
print("Duplicate reviews:", df['Text'].duplicated().sum())

display(df.head())
```

```
Shape after cleaning: (10000, 1)
Missing values: 0
Duplicate reviews: 0
```

	Text
0	I have bought several of the Vitality canned d...
1	Product arrived labeled as Jumbo Salted Peanut...
2	This is a confection that has been around a fe...
3	If you are looking for the secret ingredient i...
4	Great taffy at a great price. There was a wid...

```
for i, review in enumerate(df['Text'].head(5), 1):
    doc = nlp(review)

    print(f"\n{'='*60}")
    print(f"REVIEW {i}")
    print("Original Review:", review[:200])

    print("\nTokens:")
    print([token.text for token in doc])

    print("\nLemmas:")
    print([token.lemma_ for token in doc])

    print("\nPOS Tags:")
    print([(token.text, token.pos_) for token in doc])
```

```
Lemmas:
['I', 'have', 'buy', 'several', 'of', 'the', 'Vitality', 'can', 'dog', 'food', 'product', 'and', 'have', 'find', 'they', '

POS Tags:
[('I', 'PRON'), ('have', 'AUX'), ('bought', 'VERB'), ('several', 'ADJ'), ('of', 'ADP'), ('the', 'DET'), ('Vitality', 'PROP

============================================================
REVIEW 2
Original Review: Product arrived labeled as Jumbo Salted Peanuts...the peanuts were actually small sized unsalted. Not sur

Tokens:
['Product', 'arrived', 'labeled', 'as', 'Jumbo', 'Salted', 'Peanuts', '...', 'the', 'peanuts', 'were', 'actually', 'small'

Lemmas:
['product', 'arrived', 'label', 'as', 'Jumbo', 'Salted', 'Peanuts', '...', 'the', 'peanut', 'be', 'actually', 'small', 'si

POS Tags:
[('Product', 'NOUN'), ('arrived', 'AUX'), ('labeled', 'VERB'), ('as', 'ADP'), ('Jumbo', 'PROPN'), ('Salted', 'PROPN'), ('P

============================================================
REVIEW 3
Original Review: This is a confection that has been around a few centuries.  It is a light, pillowy citrus gelatin with nu

Tokens:
['This', 'is', 'a', 'confection', 'that', 'has', 'been', 'around', 'a', 'few', 'centuries', '.', ' ', 'It', 'is', 'a', 'li

Lemmas:
['this', 'be', 'a', 'confection', 'that', 'have', 'be', 'around', 'a', 'few', 'century', '.', ' ', 'it', 'be', 'a', 'light

POS Tags:
[('This', 'PRON'), ('is', 'AUX'), ('a', 'DET'), ('confection', 'NOUN'), ('that', 'PRON'), ('has', 'AUX'), ('been', 'AUX'),

============================================================
REVIEW 4
```

```
POS Tags:
[('If', 'SCONJ'), ('you', 'PRON'), ('are', 'AUX'), ('looking', 'VERB'), ('for', 'ADP'), ('the', 'DET'), ('secret', 'ADJ'),

============================================================
REVIEW 5
Original Review: Great taffy at a great price.  There was a wide assortment of yummy taffy.  Delivery was very quick.  If

Tokens:
['Great', 'taffy', 'at', 'a', 'great', 'price', '.', ' ', 'There', 'was', 'a', 'wide', 'assortment', 'of', 'yummy', 'taffy

Lemmas:
['great', 'taffy', 'at', 'a', 'great', 'price', '.', ' ', 'there', 'be', 'a', 'wide', 'assortment', 'of', 'yummy', 'taffy'

POS Tags:
[('Great', 'ADJ'), ('taffy', 'NOUN'), ('at', 'ADP'), ('a', 'DET'), ('great', 'ADJ'), ('price', 'NOUN'), ('.', 'PUNCT'), ('
```

```
# Analyze the first review
doc = nlp(df['Text'].iloc[0])

# Extract words by POS category
nouns = [token.text for token in doc if token.pos_ == 'NOUN']
verbs = [token.text for token in doc if token.pos_ == 'VERB']
adjectives = [token.text for token in doc if token.pos_ == 'ADJ']
adverbs = [token.text for token in doc if token.pos_ == 'ADV']

print("Nouns:", nouns)
print("\nVerbs:", verbs)
print("\nAdjectives:", adjectives)
print("\nAdverbs:", adverbs)
```

```
Nouns: ['dog', 'food', 'products', 'quality', 'product', 'stew', 'meat', 'product']

Verbs: ['bought', 'canned', 'found', 'looks', 'processed', 'smells', 'appreciates']

Adjectives: ['several', 'good', 'finicky', 'most']

Adverbs: ['more', 'better', 'better']
```

```
from collections import Counter

pos_counts = Counter()

# Analyze the first 1000 reviews
for review in df['Text'].head(1000):
    doc = nlp(review)
    for token in doc:
        if token.pos_ in ['NOUN', 'VERB', 'ADJ', 'ADV']:
            pos_counts[token.pos_] += 1

# Display frequency table
pos_df = pd.DataFrame(
    pos_counts.items(),
    columns=['POS Category', 'Frequency']
)

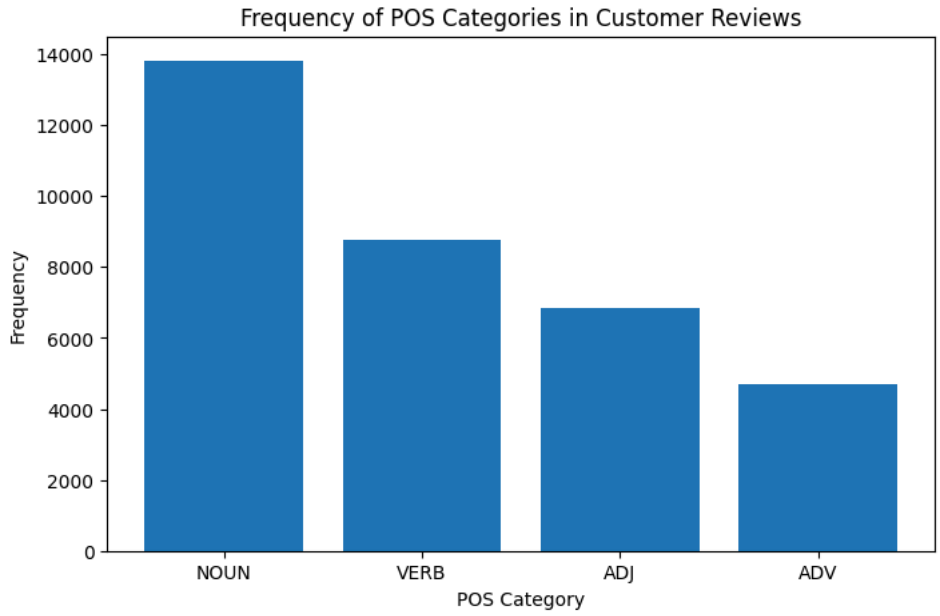
pos_df = pos_df.sort_values('Frequency', ascending=False)

display(pos_df)

# Plot POS frequencies
import matplotlib.pyplot as plt

plt.figure(figsize=(8, 5))
plt.bar(pos_df['POS Category'], pos_df['Frequency'])
plt.title('Frequency of POS Categories in Customer Reviews')
plt.xlabel('POS Category')
plt.ylabel('Frequency')
plt.show()
```

	POS Category	Frequency
2	NOUN	13808
0	VERB	8763
1	ADJ	6832
3	ADV	4709



Generate code with pos_df | New interactive sheet

```
from collections import Counter

entities = []

# Analyze the first 1000 reviews
for i, review in enumerate(df['Text'].head(1000)):
    doc = nlp(review)

    for ent in doc.ents:
        if ent.label_ in ['PERSON', 'ORG', 'DATE', 'MONEY',
                          'QUANTITY', 'PRODUCT']:
            entities.append({
                'Review No': i + 1,
                'Entity': ent.text,
                'Entity Label': ent.label_
            })

# Create a structured DataFrame
entity_df = pd.DataFrame(
    entities,
    columns=['Review No', 'Entity', 'Entity Label']
)

print("Total named entities found:", len(entity_df))
display(entity_df.head(20))

# Count entity types
if not entity_df.empty:
    print("\nEntity Label Frequencies:")
    display(entity_df['Entity Label'].value_counts().reset_index(
        name='Frequency'
    ))
else:
    print("No matching named entities found in these reviews.")
```

```
Total named entities found: 1574
```

	Review No	Entity	Entity Label
0	1	Vitality	ORG
1	1	My Labrador	PERSON
2	2	Jumbo Salted	ORG
3	3	around a few centuries	DATE
4	3	pillowy citrus gelatin	ORG
5	3	C.S. Lewis'	ORG
6	3	Edmund	PRODUCT
7	3	Sisters	PERSON
8	3	Witch	ORG
9	5	Delivery	PERSON
10	6	five pound	QUANTITY
11	6	only two weeks	DATE
12	7	Fralinger's	ORG
13	9	Wheatgrass	PERSON
14	11	Tequila Picante Gourmet de Inclan	ORG
15	13	Felidae Platinum	ORG
16	13	more than two years	DATE
17	14	Twizzlers	ORG
18	15	The Strawberry Twizzlers	ORG
19	15	Six pounds	QUANTITY

```
Entity Label Frequencies:
```

	Entity Label	Frequency
0	ORG	737
1	PERSON	378
2	DATE	297
3	MONEY	80
4	QUANTITY	61
5	PRODUCT	21

```
def extract_information(review):
    doc = nlp(review)

    return {
        'Original Review': review,
        'Tokens': [token.text for token in doc],
        'Lemmas': [token.lemma_ for token in doc],
        'POS Tags': [(token.text, token.pos_) for token in doc],
        'Named Entities': [ent.text for ent in doc.ents],
        'Entity Labels': [(ent.text, ent.label_) for ent in doc.ents]
    }


# Test with three new customer reviews
test_reviews = [
    "This chocolate tastes excellent and arrived quickly.",
    "I bought this coffee from Amazon for 15 dollars.",
    "The packaging was poor, but the product quality was amazing."
]

results = []

for review in test_reviews:
    results.append(extract_information(review))

result_df = pd.DataFrame(results)

pd.set_option('display.max_colwidth', 100)

display(result_df)
```

	Original Review	Tokens	Lemmas	POS Tags	Named Entities	Entity Labels
0	This chocolate tastes excellent and arrived quickly.	[This, chocolate, tastes, excellent, and, arrived, quickly, .]	[this, chocolate, taste, excellent, and, arrive, quickly, .]	[(This, DET), (chocolate, NOUN), (tastes, VERB), (excellent, ADJ), (and, CCONJ), (arrived, VERB)...	[]	[]
1	I bought this coffee from Amazon for 15 dollars.	[I, bought, this, coffee, from, Amazon, for, 15, dollars, .]	[I, buy, this, coffee, from, Amazon, for, 15, dollar, .]	[(I, PRON), (bought, VERB), (this, DET), (coffee, NOUN), (from, ADP), (Amazon, PROPN), (for, ADP...	[Amazon, 15 dollars]	[(Amazon, ORG), (15 dollars, MONEY)]
2	The packaging was poor, but the product quality was amazing.	[The, packaging, was, poor, ,, but, the, product, quality, was,	[the, packaging, be, poor, ,, but, the, product, quality, be,	[(The, DET), (packaging, NOUN), (was, AUX), (poor, ADJ), (,, PUNCT), (but,	[]	[]